Computation Methods

Practical Exercise - Lab Experiments

1. Write down and execute the following programs using C/C++/MATLAB/Python
2. Answer the questions at the end of each program

EXPERIMENT NO.- 1

Aim: Calculation of the value of unknown (y) corresponding to the given known (x) value using Newton Forward Formula of Interpolation.

PROGRAM:-

```
/* NEWTON FORWARD INTERPOLATION FORMULA */
#include<stdio.h>
#define MAXN 100
#define ORDER 4
void main()
{
float ax[MAXN+1],ay[MAXN+1],diff[MAXN+1][ORDER+1],nr= 1.0, dr= 1.0,x,p,h,yp; int n,i,j,k;
clrscr();
printf("Enter the values of n\n");
scanf("%d", &n);
printf("Enter the values in form x,y\n");
for (i=0; i<=n; i++)
scanf("%f %f", &ax[i],&ay[i]);
printf("Enter the values of x for which value of y is wanted \n"); scanf("%f", &x);
h = ax[1] - ax[0];
/* now making the diff. talble */
/* calculating the 1st order differences */
for (i=0; i<= n-1; i++)
diff [i][1] = ay [i+1]-ay[i];
/* calculating the second & higher order differences. */
for (j=2; j<=ORDER; j++)
for (i=0; i<=n-j; i++)
diff [i][j] = diff[i+1][j-1] - diff[i][j-1];
/* now finding x0 */
i=0;
while (!(ax[i] > x)) i++;
/* now ax[i] is x0 & ay[i] is y0 */
i-- ;
p = (x-ax[i])/h; yp=ay[i];
/* now carrying out interpolation */
for (k=1;k<=ORDER;k++)
{
```

```
nr *=p-k+1; dr *=k;
yp += (nr/dr)*diff[i][k];
}
printf ("when x = %6.1f, y = %6.2f\n", x,yp);
}
```

OUTPUT RESULT
Enter the value of n
.....
Enter the values in form of, x,y
.....
.....
.....
.....
.....
Enter the values of x for which value of y is wanted
.....
When x= ....., y=…….

EXPERIMENT No. 2

AIM: Solution of Non- Linear Equation in single variable using the method of successive bisection.

ALGORITHM:

Bisection method.
1. Decide initial values of a & b and stopping criterion, E.
2. Compute f1 =f(a) & f2 =f(b)
3. if f1 * f2 >0, a & b do not have any root and go to step 7; otherwise continue.
4. Compute *x= (a+b) /2 and compute f0 =f(*x)
5. Iff1 *f0 <0then
Set b=*x Else
Set a= *x
Set f1 = f0
6. If absolute value of (b-a)/ b is less than error E, then
root=(a+b)/2
write the value of root go to step 7
else
go to step 4
7. stop

PROGRAM:-

```
/* Bisection method. */
#include<stdio.h>
#include<conio.h>
#include<math.h>
float f(float x)
{
return(x*x*x-4*x-9); }
void bisect(float *x,float a,float b ,int *itr)
{
*x=(a+b)/2;
++(*itr);
printf("iteration no. %3d x = %7.5f\n",*itr,*x);
}
main()
{
int itr=0,maxitr;
float x,a,b,aerr,x1;
clrscr();
printf("enter the value of a,b,aloowed error,maximum iteration \n");
scanf("%f%f%f%d",&a,&b,&aerr,&maxitr); bisect(&x,a,b,&itr);
do
{
if(f(a)*f(x)<0) b=x;
else a=x;
bisect(&x1,a,b,&itr);
if (fabs(x1-x)<aerr)
{
printf("after %d iteration ,root =%6.4f\n",itr,x1);
getch();
return 0;
}
x=x1;
}
While (itr<maxtir);
Printf(“solution does not coverage,” “iteration not sufficient”);
return1;
}
```

Result:-

Enter the value of a, b, allowed error, maximum iterations

……

Iteration No. 1 x=

Iteration No. 2 x=

After iteration, root=

EXPERIMENT NO: 3

Aim: - Solution of Non – linear equation in single variable using the Regula – Falsi, Newton – Raphson method.

ALOGRITHM:

Newton – Raphson Method:

1. Assign an initial value to x, say x0
2. Evaluate f(x0) & (x0)
3. Find the improved estimation of x0
   X1 = x0 – f(x1)/f (x0)
4. Check the accuracy of the latest estimate.
   Compare relative error to a predefined value E. if [(x1 – x0)/x1]<=E Stop;
   otherwise continue.
5. Replace x0 by x1 and repeat step 3 and 4.

PROGRAM :

```
/* Regula falsi method*/
#include< stdio.h >
#include< conio.h >
#include<math.h>
float f(float x)
{
return cos (x) – x*exp(x);
}
void regula (float *x, float x0, float x1, float fx0, float fx1, int*itr)
{
*x= x0-( (x1-x0)/(fx1-fx0) ) *fx0;
++(*itr);
printf("iteration no. %3d x=%7.5f\n",*itr,*x);
}
main()
{
int itr=0,maxitr;
float x0,x1,x2, aerr, x3;
printf("enter the values for x0,x1,allowed error ,maxinum iterations\n");
scanf("%f %f %f %d ",&x0,&x1,&aerr,&maxitr);
regula(&x2,x0,x1,f(x0),f(x1),&itr);
do
{
if (f(x0)*f(x2)< 0)
```

```
x1=x2;
else
x0=x2; regula(&x3,x0,x1,f(x0),f(x1),&itr);
if(fabs(x3-x2) < aerr)
{
printf("after %d iterations,root = %6.4f\n",itr,x3);
getch();
return 0;
}
x2=x3;
}
while(itr<maxitr);
printf("solution doesnt converge,iterations not sufficient");
return 1;
}
```

Result: -
Enter the value for x0, x1, allowed error, maximum iterations. .....
Iteration No.1 x=
Iteration No.2 x=
....
After iterations, root=

EXPERIMENT NO: 4

Aim: Solution of Non – linear equation in single variable using the Newton – Raphson method.

ALOGRITHM:

Newton – Raphson Method:

1. Assign an initial value to x, say x0
2. Evaluate f(x0) & (x0)
3. Find the improved estimation of x0
   X1 = x0 – f(x1)/f (x0)
4. Check the accuracy of the latest estimate.

Compare relative error to a predefined value E. if [(x1 – x0)/x1]<=E Stop;
otherwise continue.

5. Replace x0 by x1 and repeat step 3 and 4.

PROGRAM

```
/* Newton Raphson Method*/
#include< stdio.h >
```

```
#include< conio.h >
#include<math.h>
float f(float x)
{
return x*log10(x)-1.2; }
float df(float x)
{
return log10(x)+0.43429;
}
main()
{
int itr,maxitr;
float h,x0,x1,aerr;
clrscr();
printf("enter the value of x0", "allowed errorm maximum iteration \n");
scanf("%f%f%d",&x0,&aerr,&maxitr);
for(itr=1;itr<=maxitr;itr++)
{
h=f(x0)/df(x0);
x1=x0-h;
printf("iteration %3d,x=%9.6f\n",itr,x1);
if(fabs(h)<aerr)
{
printf("after %3d iteration,root=%8.6f\n",itr,x1);
return 0;
}
x0=x1;
}
printf("solution does not converge," "iteration not sufficient ");
return 1;
}
```

Result:

Enter the x0, allowed error, maximum iterations.
……..
Iteration No.1 x=
Iteration No.2 x=
....
After iterations, root=

EXPERIMENT NO: 5

Aim: Solution of a system of simultaneous algebraic equation using Gaussian elimination procedure.

ALOGRITHM:

1. Arrange equation such that a11 !=0
2. Eliminate x1 from all but the first equation. This is done as follows:
   (i) Normalilse the first equation by dividing it by a11.
   (ii) Subtract from the second equation a21 times the normalized first equation. The result is [a21-a21*(a11/a11]x1+[a22-a21*(a12/a11]x2+.....=b2-a21*(b11/a11) a21-a21*(a11/a11)=0
   Thus, the resultant equation does not contain x1. the new second equation is 0+a'22x2+....+a'2nxn=b'2
   (iii) Similarly, subtract from the third equation. a31 times the normalized first equation. The result would be

0+a'32x2+.....+a'3nXn=b'3

if we repeat this procedure till the nth equation is operated on, we will get the following new system of equation:

a11x1+a12x2+.....+a1xn=b1
a'22x2+.....+a'2nxn=b'2
...
...
...
a'n2x2+....+a'nnxnn=b'n

The solution of these equation is same as that of the original equation 3.

3. Eliminate x2 from the third to last equation in the new set.
   Again, we assume that a'22!=0
   (i) Subtract from the third equation a'32 times the normalized second equation.
   (ii) Subtract from the fourth equation, a'42 times the normalized second equation and so on .

This process will continue till the last equation contains only one unknown, namely, xn. The final form of the equations will look like this:

a11 x1+a12 x2+...+a1n xn=b1 a'22x2+....+a'2n xn+b'2
.....
.....

This process is called triangulation. The number of primes indicate the number of times the coefficient has been modified.

4. Obtain solution by back substitution. The solution is as follows:
   Xn=bn(n-1)/ann(n-1)

This can be subsitituted back in the (n-1) th equation to obtain the solution for xn-1. This back substitution can be continued till we get the solution for x1.

PROGRAM:

```
/*gauss elimination method*/
#include<stdio.h>
#define N 4
main()
{
float a[N][N+1],x[N],t,s;
int i,j,k;
clrscr();
printf("enter the elements of the augmented matrix rowwise\n");
for(i=0;i<N;i++)
for(j=0;j<N+1;j++)
scanf("%f",&a[i][j]);
for(j=0;j<N-1;j++)
for(i=j+1;i<N;i++)
{ t=a[i][j]/a[j][j];
for(k=0;k<N+1;k++)
a[i][k]-=a[j][k]*t;
}
/*now print the upper triangular matrix*/
printf("the upper triangular matrix is \n");
for(i=0;i<N;i++)
{
for(j=0;j<N+1;j++)
printf("%8.4f",a[i][j]);
printf("\n");
}
/*now performing back substitution*/
for(i=N-1;i>=0;i--)
{ s=0;
for(j=i+1;j<N;j++)
s+=a[i][j]*x[j];
x[i]=(a[i][j]-s)/a[i][i];
}
/*now printing the result*/
printf("sol is \n"); for(i=0;i<N;i++)
printf("x[%3d]=%8.4f\n",i+1,x[i]);
getch();
}
```

Result: -

Enter the elements of augmented matrix row wise.

....

The upper triangular matrix is: - ....

The solution is:

X[1]=
X[2]=
....
....
....
X[n]=

EXPERIMENT NO: 6

AIM: - Calculation of integral value of a given function using Trapezoidal Rule of Numerical Integration

PROGRAM:

```
/* Trapezoidal rule. */
#include<stdio.h>
float y(float x)
{
return 1/(1+x*x); }
void main()
{
float x0,xn,h,s;
int i,n;
puts("Enter x0,xn,no. of subintervals");
scanf("%f %f %d", &x0,&xn,&n);
h = (xn - x0)/n;
s = y(x0)+y(xn);
for (i = 1; i<=n-1; i++)
s += 2*y(x0+i*h);
printf ("Value fo integral is = % 6.4f\n",(h/2)*s);
}
```

OUTPUT RESULT

Enter x0, xn, no. of subintervals .....
Value of integral is = ...